

## confest.py

```
import pytest
from sqlalchemy import create_engine
from sqlalchemy.orm import sessionmaker
from fastapi.testclient import TestClient
from app.main import app
from app.db.database import Base, get_db

SQLALCHEMY_DATABASE_URL = "sqlite:///./test.db"

engine = create_engine(
    SQLALCHEMY_DATABASE_URL,
    connect_args={"check_same_thread": False}
)

TestingSessionLocal = sessionmaker(
    autocommit=False,
    autoflush=False,
    bind=engine
)

@pytest.fixture
def db():
    Base.metadata.drop_all(bind=engine)
    Base.metadata.create_all(bind=engine)

    db = TestingSessionLocal()
    try:
        yield db
    finally:
        db.close()

@pytest.fixture
def client(db):
    def override_get_db():
        yield db

    app.dependency_overrides[get_db] = override_get_db

    client = TestClient(app)
    yield client
    app.dependency_overrides.clear()
```

import pytest → importa pytest. create\_engine → crea conexión a la DB. sessionmaker → crea sesiones. TestClient → simula peticiones HTTP. app → tu aplicación FastAPI. Base → modelos. get\_db → dependencia original. SQLALCHEMY\_DATABASE\_URL → DB de test. engine → crea motor SQLite. check\_same\_thread=False → permite multihilo. TestingSessionLocal → fábrica de sesiones. @pytest.fixture db → fixture de base de datos. drop\_all → borra tablas. create\_all → crea tablas nuevas. db = TestingSessionLocal() → crea sesión. yield db → entrega DB al test. db.close() → cierra sesión. @pytest.fixture client → fixture cliente HTTP. override\_get\_db → reemplaza DB real por test. dependency\_overrides → aplica override. TestClient(app) → cliente HTTP. yield client → entrega cliente. clear() → limpia overrides.

## test\_auth.py

```
def test_register(client):
    response = client.post("/users/register", json={
        "email": "test1@test.com",
        "password": "123456"
    })

    assert response.status_code == 200
    assert response.json()["email"] == "test1@test.com"
```

def test\_register → define test. client → fixture inyectado. client.post → hace petición HTTP.  
json={...} → body request. assert status\_code == 200 → verifica éxito. response.json() → obtiene  
respuesta JSON. ["email"] → accede al campo email.

## test\_expenses.py

```
def test_create_expense(client):  
    token = create_user_and_login(client, email="user1@test.com")  
  
    response = create_expense(client, token)  
  
    assert response.status_code == 200  
    assert response.json()["amount"] == 10
```

def test\_create\_expense → test creación gasto. create\_user\_and\_login → crea usuario + token.  
token → JWT para autenticación. create\_expense → crea gasto vía API. assert status\_code →  
verifica éxito. response.json() → datos devueltos. ["amount"] → comprueba valor.

## utils.py

```
def create_user_and_login(client, email="test@test.com", password="123456"):  
    client.post("/users/register", json={  
        "email": email,  
        "password": password  
    })  
  
    response = client.post("/users/login", json={  
        "email": email,  
        "password": password  
    })  
  
    return response.json()["access_token"]
```

def create\_user\_and\_login → función auxiliar. client.post register → crea usuario. json → datos  
enviados. client.post login → login usuario. response → guarda respuesta. response.json() →  
obtiene JSON. ["access\_token"] → devuelve token.